M.Sc. Lab Assignment – Introduction to Quantum Gates using Qiskit

Course: Quantum Computing / Quantum Machine Learning
Class: M.Sc.
Lab Duration: 2 Hours
Lab Title: Study and Implementation of Quantum Gates using Qiskit

Objective
To understand basic quantum gates, install the Qiskit library, and implement simple quantum circuits using Python.

Part 1: Theoretical Study

Pauli-X gate:

Circuit symbol
——[X]——

Matrix Representation:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Effect on Basis States

- $X|0\rangle = |1\rangle$
- $X|1\rangle = |0\rangle$

The Pauli-X gate works like a classical NOT gate.
It flips the qubit:

- 0 becomes 1
- 1 becomes 0

That is why it is also called the quantum bit-flip gate.

Pauli-Y gate:

Circuit Symbol :
——[Y]——

Matrix Representation:

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Effect on Basis States

- $Y|0\rangle = i|1\rangle$
- $Y|1\rangle =- i|0\rangle$

The Pauli-Y gate flips the qubit like the X gate, but it also adds a phase factor (i).
So it performs:

- Bit flip
- Plus a phase rotation

It is sometimes called a bit and phase flip gate.

Pauli-Z gate:

Circuit Symbol:

——[Z]——

Matrix Representation:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Effect on Basis States

- $Z|0\rangle = |0\rangle$
- $Z|1\rangle = -|1\rangle$

The Pauli-Z gate does not change 0.
But it changes the phase of 1 by adding a negative sign.
So:

- $|0\rangle$ stays same
- $|1\rangle$ becomes $-|1\rangle$

This is called a phase-flip gate.

Hadamard (H) Gate:

Circuit Symbol:

——[H]——

Matrix Representation:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Effect on Basis States

- $H|0\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$
- $H|1\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}$

The Hadamard gate creates a superposition.
It turns:

- $|0\rangle$ into equal mix of $|0\rangle$ and $|1\rangle$
- $|1\rangle$ into equal mix but with a negative phase

CNOT Gate (Controlled-NOT):

Circuit Symbol:

```
Control ——●——
          |
Target  ——⊕——
```

Matrix Representation:

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Effect on Basis States

- $|00\rangle \rightarrow |00\rangle$
- $|01\rangle \rightarrow |01\rangle$
- $|10\rangle \rightarrow |11\rangle$
- $|11\rangle \rightarrow |10\rangle$

The CNOT gate works on two qubits:

- The first qubit is the control
- The second qubit is the target

If control = 1 ⟶ flip the target
If control = 0 ⟶ do nothing
This gate is very important because it creates entanglement.

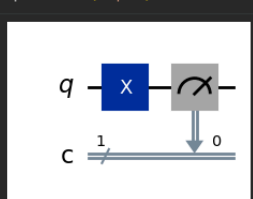
Part 2: Implementation of Quantum Gates

```
[14]
✓ 0s
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
%matplotlib inline
```

```
[9]
from qiskit_aer import AerSimulator

# Create simulator
simulator = AerSimulator()
```

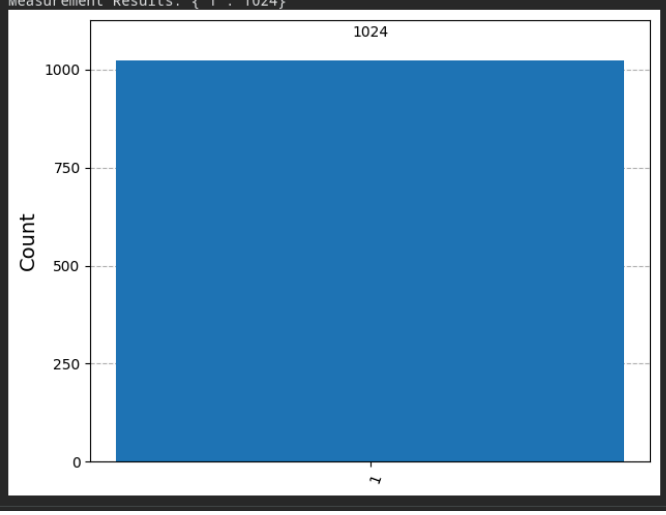
1. Create a single qubit circuit and apply X gate.

```
# Create circuit (1 qubit, 1 classical bit)
qc_x = QuantumCircuit(1, 1)

# Apply X gate
qc_x.x(0)

# Measurement
qc_x.measure(0, 0)

# Display circuit diagram
qc_x.draw('mpl')
```



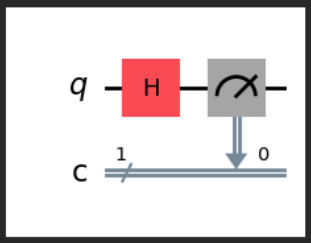
```python
compiled_circuit = transpile(qc_x, simulator)
job = simulator.run(compiled_circuit, shots=1024)

result = job.result()
counts = result.get_counts()

print("Measurement Results:", counts)

# Store figure first
fig = plot_histogram(counts)

# Display explicitly
display(fig)
```

```
Measurement Results: {'1': 1024}
```



2. Create a single qubit circuit and apply an H gate.

```python
qc_h = QuantumCircuit(1, 1)

# Apply Hadamard gate
qc_h.h(0)

# Measurement
qc_h.measure(0, 0)

# Circuit diagram
qc_h.draw('mpl')
```

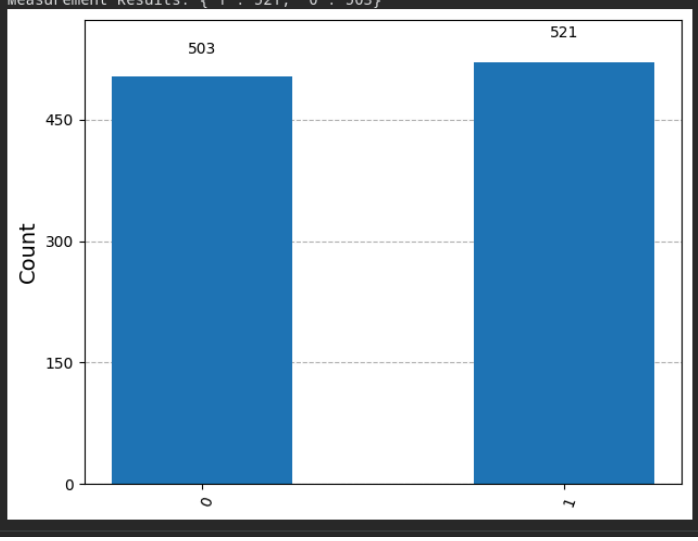



```python
compiled_circuit = transpile(qc_h, simulator)
job = simulator.run(compiled_circuit, shots=1024)

result = job.result()
counts = result.get_counts()

print("Measurement Results:", counts)
```

```
Measurement Results: {'1': 514, '0': 510}
```

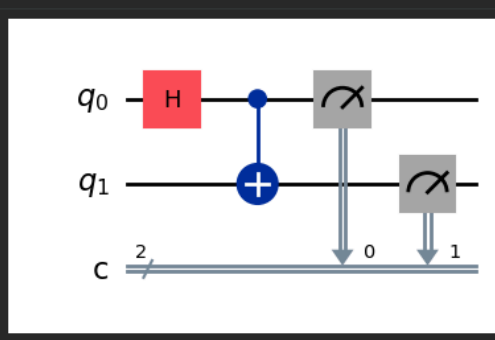
```
compiled_circuit = transpile(qc_h, simulator)
job = simulator.run(compiled_circuit, shots=1024)

result = job.result()
counts = result.get_counts()

print("Measurement Results:", counts)

# Store figure first
fig = plot_histogram(counts)

# Display explicitly
display(fig)
```

```
Measurement Results: {'1': 521, '0': 503}
```



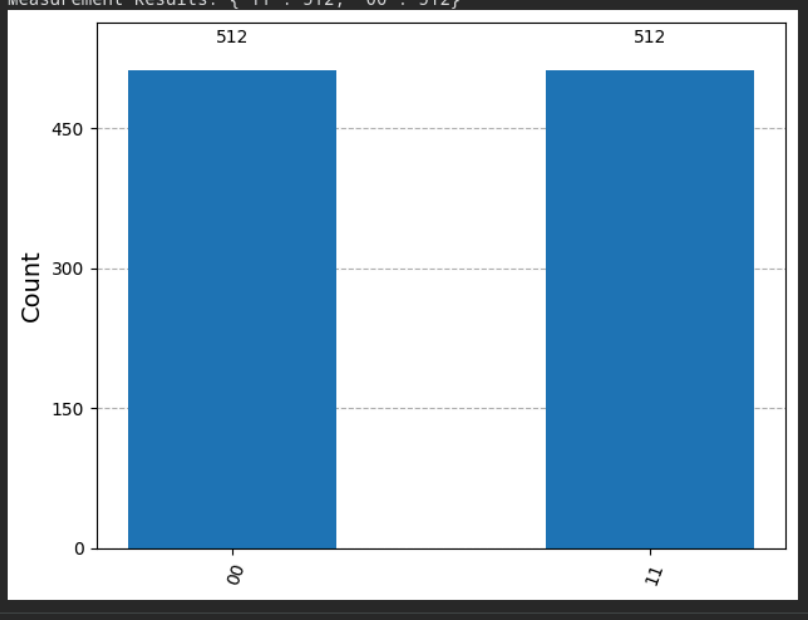
3. Create a two-qubit circuit and apply a CNOT gate.

```
qc_cnot = QuantumCircuit(2, 2)

# Put first qubit in superposition
qc_cnot.h(0)

# Apply CNOT
qc_cnot.cx(0, 1)

# Measurement
qc_cnot.measure([0,1], [0,1])

# Circuit diagram
qc_cnot.draw('mpl')
```



```python
compiled_circuit = transpile(qc_cnot, simulator)
job = simulator.run(compiled_circuit, shots=1024)

result = job.result()
counts = result.get_counts()

print("Measurement Results:", counts)

# Store figure first
fig = plot_histogram(counts)

# Display explicitly
display(fig)
```

```
Measurement Results: {'11': 512, '00': 512}
```



Part 4: Analysis Questions

1. What happens when the H gate is applied twice?

Ans: When the Hadamard (H) gate is applied two times in a row, the qubit returns to its original state.

- First H creates superposition.
- Second H cancels that superposition.
- So applying H twice is like doing nothing.

It acts like a reversible operation.

2. What is superposition?

Ans- Superposition is a quantum property where a qubit can exist in a combination of both $|0\rangle$ and $|1\rangle$ at the same time.

A general qubit state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Where:

- $\alpha$and β are probability amplitudes
- $|\alpha|^2 + |\beta|^2 = 1$

Unlike a classical bit (which is either 0 or 1),
a qubit can be partly 0 and partly 1 until measured.

When measured:

- It collapses to either 0 or 1.

Superposition is the reason quantum computers can process many possibilities at once.

3. How does CNOT create entanglement?

The CNOT gate entangles two qubits when:

- The control qubit is first put into superposition (usually using H gate).
- Then CNOT is applied.
- Example:
- Step 1:

$$|00\rangle$$

- Step 2 (apply H to first qubit):

$$\frac{|00\rangle+|10\rangle}{\sqrt{2}}$$

- Step 3 (apply CNOT):

$$\frac{|00\rangle+|11\rangle}{\sqrt{2}}$$

- Now the two qubits are entangled.

- Why is this entanglement?

Because:

You cannot describe one qubit independently.

If you measure the first qubit and get $0 \rightarrow$ second is also 0.

If you measure the first qubit and get $1 \rightarrow$ second is also 1.

Their results are correlated.

4. Why are quantum gates reversible?

Quantum gates are reversible because they are represented by **unitary matrices**.

A unitary matrix Usatisfies:

$$U^{\dagger}U = I$$

This means:

- Every quantum operation has an inverse.
- No information is destroyed.

- You can always reverse the operation.

In classical computing: Some operations lose information (like AND gate).

In quantum computing:

- Information cannot be lost.
- So every gate must be reversible.

Example:

- X gate is its own inverse.
- H gate is its own inverse.
- CNOT is also reversible.